Deep Learning and AI Frameworks

Huawei HCIA-AI V3.5 Endterm Implementation Report

Author: Angsar Shaumen

Instructor: Akhmetova Zhanar

Subject: Artificial Intelligence and Neural Networks

Astana IT University
May 2, 2026

Contents

Contents **1**

Course Completion & Code Repository **2**

1 AI Final Exam Lab (MNIST Digit Recognition) **3**
1.1 Huawei AI Certification: AI Final Exam Lab 3

2 ModelArts Transfer Learning (MobileNetV2) **14**
2.1 Huawei AI Certification: Lab 9.5 14

Course Completion & Code Repository

The entirety of the source code, executed Jupyter Notebooks, and dataset configurations produced for the Endterm evaluation can be found on our GitHub Repository:

https://github.com/ansinitro/ai-nn/tree/main/midterm/Labs_notebooks

This document strictly encompasses the final two major laboratories: the comprehensive AI Final Exam evaluating standard Deep Neural Networks against Spatial Convolutional Neural Networks on the MNIST handwritten digit database, and the Image Classification Pipeline (ModelArts Alternative) leveraging computationally intensive Transfer Learning via MobileNetV2 on the Flower Dataset.

Chapter 1

AI Final Exam Lab (MNIST Digit Recognition)

1.1 Huawei AI Certification: AI Final Exam Lab

Handwritten Digit Recognition (MNIST)

Objective: Master the basic process of TensorFlow computing and build neural networks to classify MNIST hand-written digits. We will evaluate a Deep Neural Network (DNN) against a Convolutional Neural Network (CNN) architecture.

```
import os
import numpy as np
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import datasets, layers, optimizers
from tensorflow.keras.layers import Conv2D, MaxPool2D,
   Dropout, Flatten, Dense
from tensorflow.keras.models import Sequential, load_model

# Ensure inline plotting
%matplotlib inline
```

```
WARNING: All log messages before absl::InitializeLog() is
   called are written to STDERR
I0000 00:00:1777685359.528055 2702417 cudart_stub.cc:31]
   Could not find cuda drivers on your machine, GPU will not
   be used.
I0000 00:00:1777685359.587772 2702417 cpu_feature_guard.cc
   :227] This TensorFlow binary is optimized to use available
    CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other
   operations, rebuild TensorFlow with the appropriate
   compiler flags.
```

```
WARNING: All log messages before absl::InitializeLog() is
   called are written to STDERR
I0000 00:00:1777685361.442989 2702417 cudart_stub.cc:31]
   Could not find cuda drivers on your machine, GPU will not
   be used.
```

1. Dataset Acquisition

Load the MNIST dataset natively into training and test variables, and construct one-hot encoded tensors for classification.

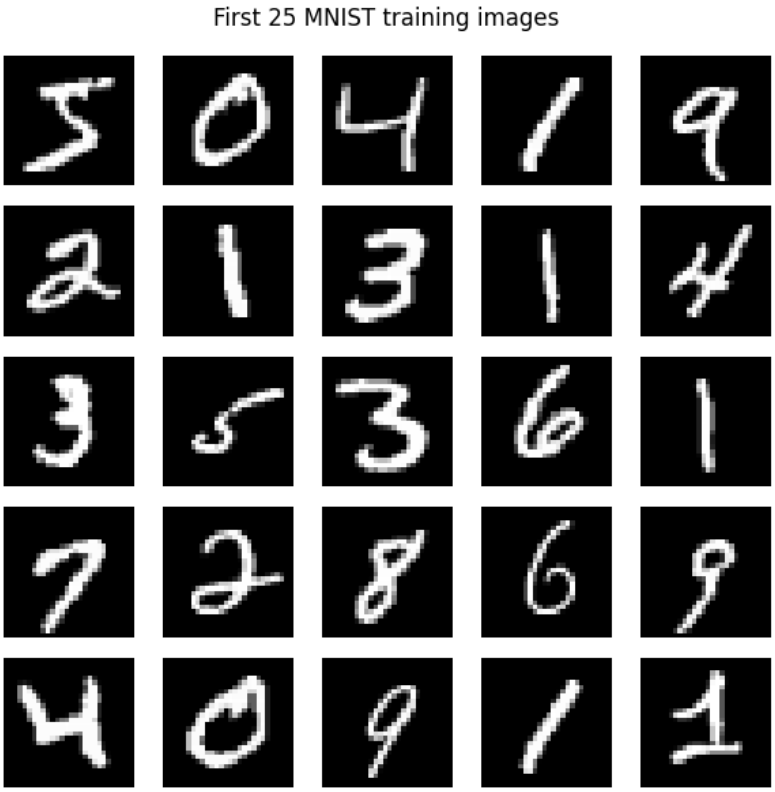
```
(x_train, y_train), (x_test, y_test) = datasets.mnist.
   load_data()
print("First 5 labels:", y_train[:5])
print("x_train shape:", x_train.shape, "y_train shape:",
   y_train.shape)
print("x_test shape :", x_test.shape, "y_test shape :",
   y_test.shape)

n_classes = 10
y_train_oh = keras.utils.to_categorical(y_train, n_classes)
y_test_oh = keras.utils.to_categorical(y_test, n_classes)
print("One-hot sample:\n", y_train_oh[:5])
```

```
First 5 labels: [5 0 4 1 9]
x_train shape: (60000, 28, 28) y_train shape: (60000,)
x_test shape : (10000, 28, 28) y_test shape : (10000,)
One-hot sample:
 [[0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]]
```

2. Data Visualization

Extract and plot exactly how the computer natively parses the numeric pixel grids.

```
plt.figure(figsize=(6, 6))
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.axis("off")
plt.suptitle("First 25 MNIST training images")
plt.tight_layout()
plt.show()
```



3. Deep Neural Network (DNN) Construction

We flat-pack the 28x28 matrices into 1D (784) tensors, and map them linearly across multiple dense feed-forward node layers.

```
# Preprocess for DNN (flatten + normalize)
X_train_dnn = x_train.reshape(60000, 784).astype("float32") /
    255.0
X_test_dnn = x_test.reshape(10000, 784).astype("float32") /
   255.0

dnn = Sequential([
    Dense(512, activation="relu", input_dim=784),
    Dense(256, activation="relu"),
    Dense(128, activation="relu"),
    Dense(n_classes, activation="softmax"),
])
dnn.summary()

dnn.compile(
    loss=keras.losses.categorical_crossentropy,
    optimizer=optimizers.Adam(0.001),
    metrics=["accuracy"],
)

dnn_history = dnn.fit(
    X_train_dnn, y_train_oh,
```

```
    batch_size=128, epochs=10, verbose=2,
    validation_data=(X_test_dnn, y_test_oh),
)

dnn_score = dnn.evaluate(X_test_dnn, y_test_oh, verbose=0)
print("DNN␣Test␣loss:", dnn_score[0])
print("DNN␣Test␣accuracy:", dnn_score[1])
```

```
/home/ansinitro/AITU/ai-nn/midterm/.venv/lib/python3.12/site-
   packages/keras/src/layers/core/dense.py:107: UserWarning:
   Do not pass an `input_shape`/`input_dim` argument to a
   layer. When using Sequential models, prefer using an `
   Input(shape)` object as the first layer in the model
   instead.
  super().__init__(activity_regularizer=activity_regularizer,
      **kwargs)
E0000 00:00:1777685363.240408 2702417 cuda_executor.cc:1737]
   INTERNAL: CUDA Runtime error: Failed call to
   cudaGetRuntimeVersion: Error loading CUDA libraries. GPU
   will not be used.: Error loading CUDA libraries. GPU will
   not be used.
W0000 00:00:1777685363.240779 2702502 cuda_executor.cc:1755]
   Failed to determine cuDNN version (Note that this is
   expected if the application doesn't link the cuDNN plugin)
   : INTERNAL: cuDNN error: CUDNN_STATUS_INTERNAL_ERROR
W0000 00:00:1777685363.255852 2702417 gpu_device.cc:2365]
   Cannot dlopen some GPU libraries. Please make sure the
   missing libraries mentioned above are installed properly
   if you would like to use GPU. Follow the guide at https://
   www.tensorflow.org/install/gpu for how to download and
   setup the required libraries for your platform.
Skipping registering GPU devices...
```

```
Epoch 1/10


W0000 00:00:1777685363.614474 2702417 cpu_allocator_impl.cc
   :82] Allocation of 188160000 exceeds 10% of free system
   memory.


469/469 - 4s - 10ms/step - accuracy: 0.9319 - loss: 0.2288 -
   val_accuracy: 0.9677 - val_loss: 0.1055
Epoch 2/10
469/469 - 3s - 6ms/step - accuracy: 0.9737 - loss: 0.0863 -
   val_accuracy: 0.9723 - val_loss: 0.0853
Epoch 3/10
469/469 - 3s - 6ms/step - accuracy: 0.9827 - loss: 0.0555 -
   val_accuracy: 0.9770 - val_loss: 0.0739
```

```
Epoch 4/10
469/469 - 3s - 6ms/step - accuracy: 0.9876 - loss: 0.0391 -
   val_accuracy: 0.9724 - val_loss: 0.0953
Epoch 5/10
469/469 - 3s - 6ms/step - accuracy: 0.9893 - loss: 0.0317 -
   val_accuracy: 0.9754 - val_loss: 0.0865
Epoch 6/10
469/469 - 3s - 7ms/step - accuracy: 0.9912 - loss: 0.0272 -
   val_accuracy: 0.9789 - val_loss: 0.0750
Epoch 7/10
469/469 - 3s - 7ms/step - accuracy: 0.9934 - loss: 0.0205 -
   val_accuracy: 0.9789 - val_loss: 0.0769
Epoch 8/10
469/469 - 4s - 8ms/step - accuracy: 0.9928 - loss: 0.0217 -
   val_accuracy: 0.9798 - val_loss: 0.0792
Epoch 9/10
469/469 - 3s - 7ms/step - accuracy: 0.9940 - loss: 0.0179 -
   val_accuracy: 0.9780 - val_loss: 0.0850
Epoch 10/10
469/469 - 3s - 7ms/step - accuracy: 0.9953 - loss: 0.0142 -
   val_accuracy: 0.9799 - val_loss: 0.0817
DNN Test loss: 0.08167005330324173
DNN Test accuracy: 0.9799000024795532
```

4. Convolutional Neural Network (CNN) Construction

We preserve the spatial 2D matrix shape, allowing the CNN kernels to systematically extract positional hierarchies and edge gradients natively, maximizing feature recognition accuracy!

```
X_train_cnn = x_train.reshape(-1, 28, 28, 1).astype("float32"
   ) / 255.0
X_test_cnn = x_test.reshape(-1, 28, 28, 1).astype("float32")
   / 255.0

cnn = Sequential()
cnn.add(Conv2D(32, kernel_size=(3, 3), padding="same",
   activation="relu", input_shape=(28, 28, 1)))
cnn.add(MaxPool2D(pool_size=(2, 2)))
cnn.add(Conv2D(64, kernel_size=(3, 3), padding="valid",
   activation="relu"))
cnn.add(MaxPool2D(pool_size=(2, 2)))
cnn.add(Dropout(0.25))
cnn.add(Flatten())
cnn.add(Dense(128, activation="relu"))
cnn.add(Dropout(0.5))
cnn.add(Dense(n_classes, activation="softmax"))
cnn.summary()
```

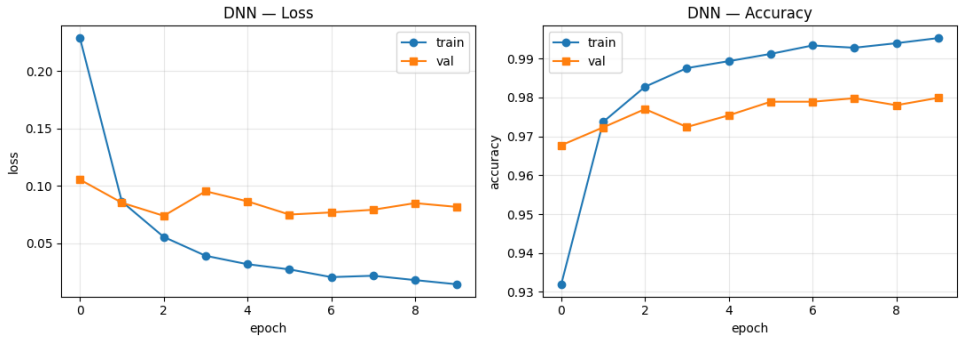
```
cnn.compile(
    loss="categorical_crossentropy",
    optimizer="adam",
    metrics=["accuracy"],
)

cnn_history = cnn.fit(
    X_train_cnn, y_train_oh,
    epochs=5, batch_size=128, verbose=2,
    validation_data=(X_test_cnn, y_test_oh),
)

cnn_score = cnn.evaluate(X_test_cnn, y_test_oh, verbose=0)
print("CNN␣Test␣Loss:", cnn_score[0])
print("CNN␣Test␣Accuracy:", cnn_score[1])
```

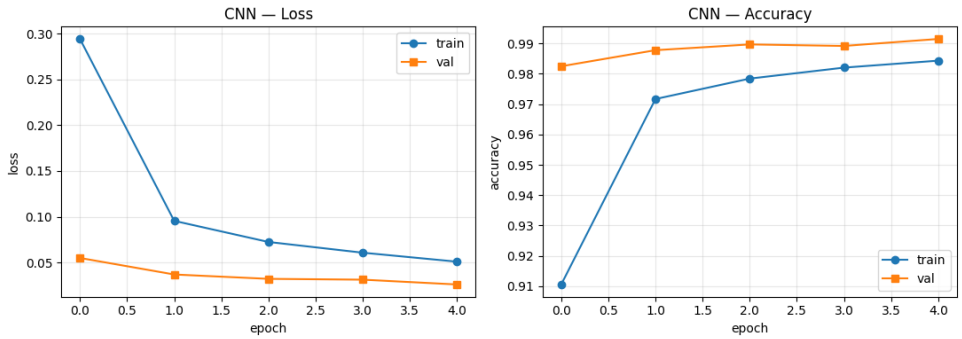
```
/home/ansinitro/AITU/ai-nn/midterm/.venv/lib/python3.12/site-
   packages/keras/src/layers/convolutional/base_conv.py:113:
   UserWarning: Do not pass an `input_shape`/`input_dim`
   argument to a layer. When using Sequential models, prefer
   using an `Input(shape)` object as the first layer in the
   model instead.
  super().__init__(activity_regularizer=activity_regularizer,
      **kwargs)
```

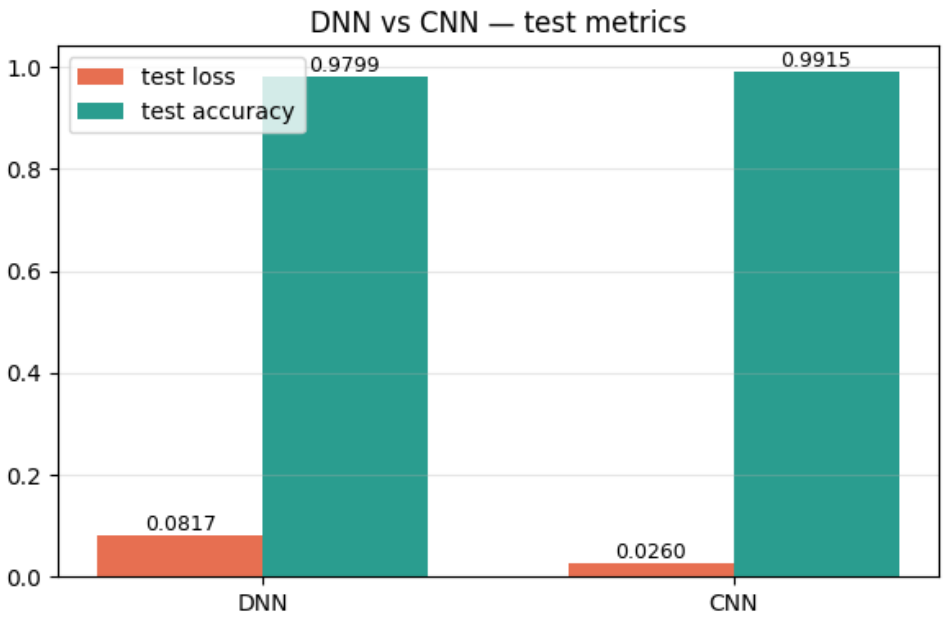
```
Epoch 1/5


W0000 00:00:1777685397.829101 2702417 cpu_allocator_impl.cc
   :82] Allocation of 188160000 exceeds 10% of free system
   memory.


469/469 - 21s - 45ms/step - accuracy: 0.9106 - loss: 0.2942 -
    val_accuracy: 0.9825 - val_loss: 0.0550
Epoch 2/5
469/469 - 17s - 37ms/step - accuracy: 0.9717 - loss: 0.0955 -
    val_accuracy: 0.9878 - val_loss: 0.0369
Epoch 3/5
469/469 - 18s - 38ms/step - accuracy: 0.9784 - loss: 0.0724 -
    val_accuracy: 0.9897 - val_loss: 0.0321
Epoch 4/5
469/469 - 16s - 35ms/step - accuracy: 0.9821 - loss: 0.0607 -
    val_accuracy: 0.9892 - val_loss: 0.0312
Epoch 5/5
469/469 - 16s - 34ms/step - accuracy: 0.9844 - loss: 0.0509 -
    val_accuracy: 0.9915 - val_loss: 0.0260
CNN Test Loss: 0.026009762659668922
CNN Test Accuracy: 0.9915000200271606
```

5. Architectural Evaluation and Training Validation Curves

```
def plot_history(history, title):
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    axes[0].plot(history.history["loss"], "o-", label="train"
       )
    axes[0].plot(history.history["val_loss"], "s-", label="
       val")
    axes[0].set_title(f"{title}␣—␣Loss")
    axes[0].set_xlabel("epoch"); axes[0].set_ylabel("loss")
    axes[0].legend(); axes[0].grid(alpha=0.3)

    axes[1].plot(history.history["accuracy"], "o-", label="
       train")
    axes[1].plot(history.history["val_accuracy"], "s-", label
       ="val")
    axes[1].set_title(f"{title}␣—␣Accuracy")
    axes[1].set_xlabel("epoch"); axes[1].set_ylabel("accuracy
       ")
    axes[1].legend(); axes[1].grid(alpha=0.3)

    plt.tight_layout()
    plt.show()

plot_history(dnn_history, "DNN")
plot_history(cnn_history, "CNN")

# ---------- Model comparison bar chart ----------
fig, ax = plt.subplots(figsize=(6, 4))
names = ["DNN", "CNN"]
losses = [dnn_score[0], cnn_score[0]]
accs = [dnn_score[1], cnn_score[1]]
xs = np.arange(len(names))
w = 0.35
b1 = ax.bar(xs - w / 2, losses, w, label="test␣loss", color="
   #e76f51")
b2 = ax.bar(xs + w / 2, accs, w, label="test␣accuracy", color
   ="#2a9d8f")
ax.set_xticks(xs); ax.set_xticklabels(names)
ax.set_title("DNN␣vs␣CNN␣—␣test␣metrics")
ax.legend(); ax.grid(axis="y", alpha=0.3)
for b in (*b1, *b2):
    ax.text(b.get_x() + b.get_width() / 2, b.get_height(), f"
       {b.get_height():.4f}", ha="center", va="bottom",
       fontsize=9)
plt.tight_layout()
plt.show()
```





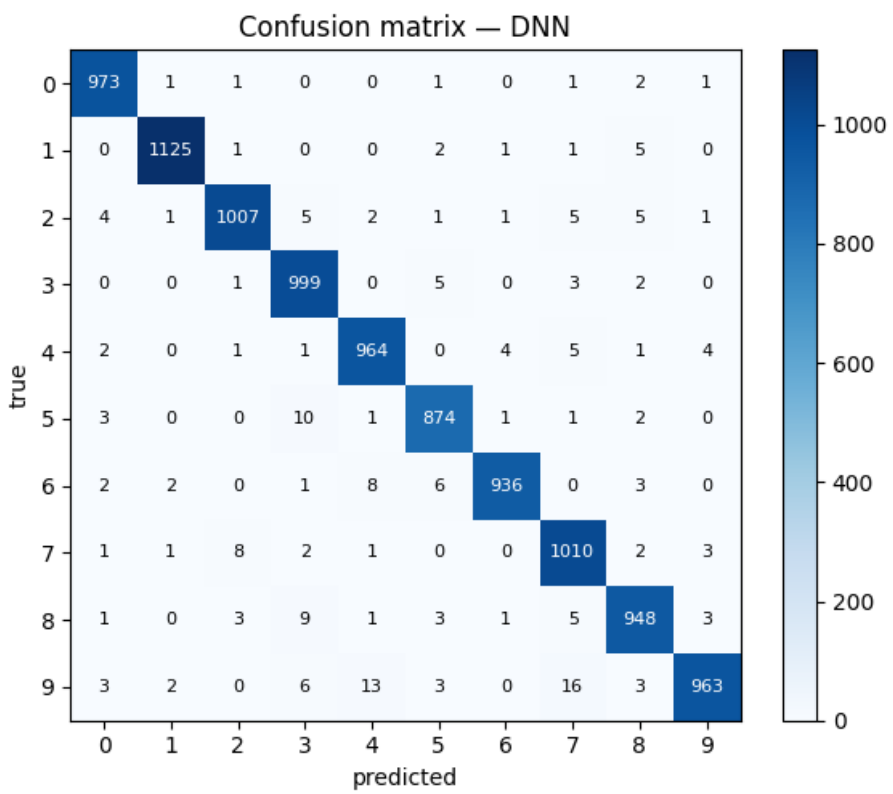


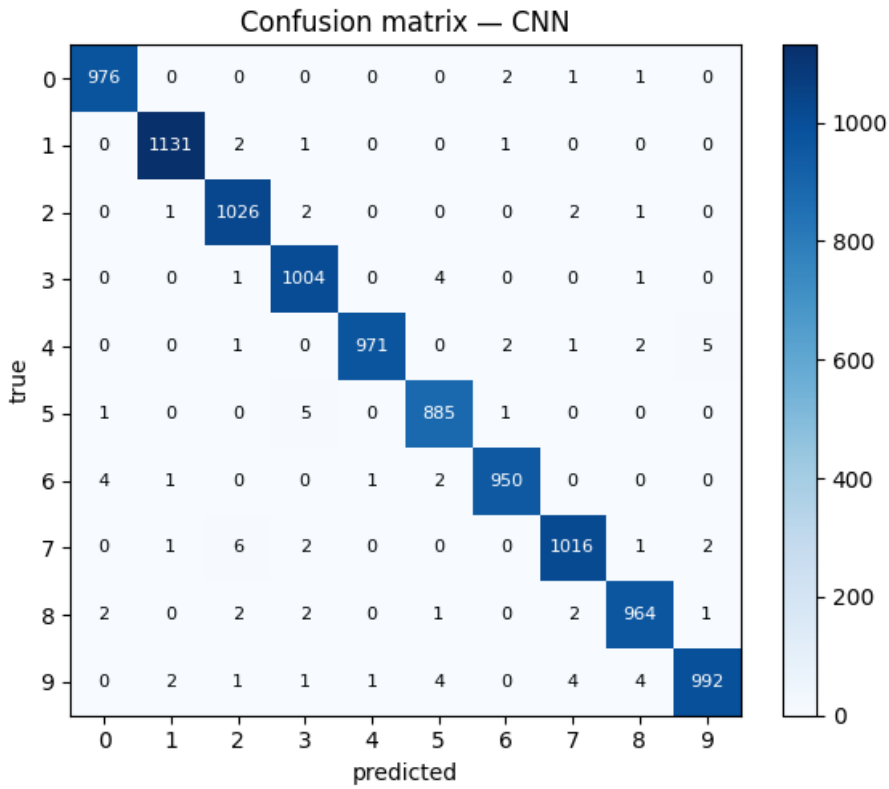
6. Confusion Matrices and Predictions

Finally, evaluating the exact false-positive failure modes between architectures natively.

```
def plot_confusion(model, X, y_true, title):
    probs = model.predict(X, verbose=0)
    y_pred = np.argmax(probs, axis=1)
```

```
    cm = np.zeros((n_classes, n_classes), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1

    fig, ax = plt.subplots(figsize=(6, 5))
    im = ax.imshow(cm, cmap="Blues")
    ax.set_title(f"Confusion matrix — {title}")
    ax.set_xlabel("predicted"); ax.set_ylabel("true")
    ax.set_xticks(range(n_classes)); ax.set_yticks(range(
       n_classes))
    fig.colorbar(im, ax=ax)
    thresh = cm.max() / 2.0
    for i in range(n_classes):
        for j in range(n_classes):
            ax.text(j, i, cm[i, j], ha="center", va="center",
                fontsize=8, color="white" if cm[i, j] >
               thresh else "black")
    plt.tight_layout()
    plt.show()
    return y_pred

dnn_pred = plot_confusion(dnn, X_test_dnn, y_test, "DNN")
cnn_pred = plot_confusion(cnn, X_test_cnn, y_test, "CNN")
```



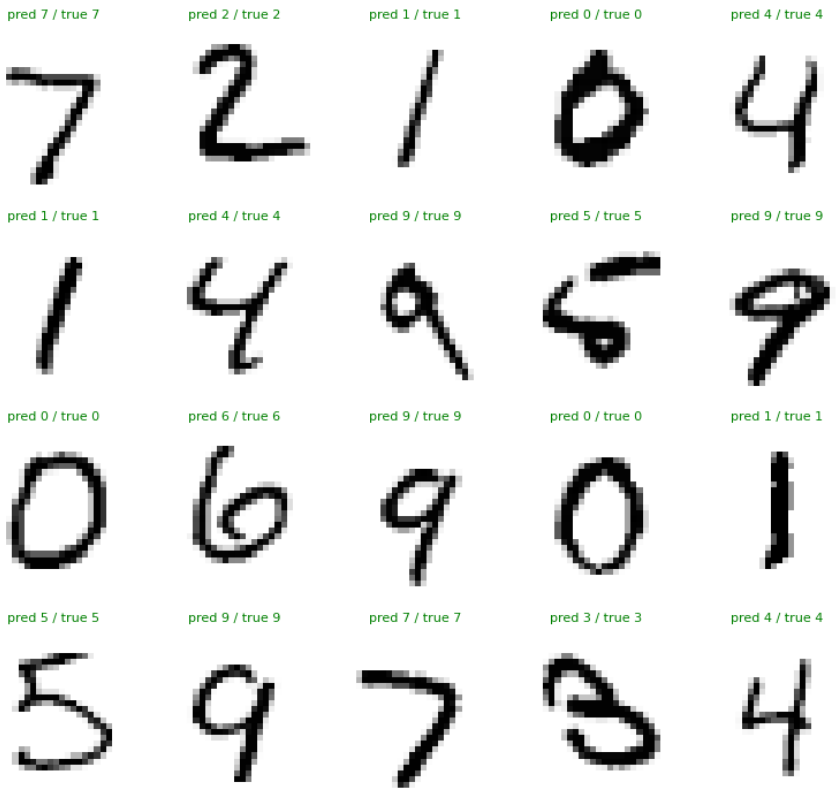


Model Deployment Simulation

Running a mock inference pipeline on 20 raw tensor inputs to determine deployment readiness.

```
def res_visual(n):
    probs = cnn.predict(X_test_cnn[0:n], verbose=0)
    final_opt_a = np.argmax(probs, axis=1)

    fig, ax = plt.subplots(nrows=int(n / 5), ncols=5, figsize
       =(8, 1.8 * (n // 5)))
    ax = ax.flatten()
    print(f"Prediction␣results␣of␣the␣first␣{n}␣images:")
    for i in range(n):
        img = X_test_cnn[i].reshape((28, 28))
        ax[i].imshow(img, cmap="Greys")
        ok = final_opt_a[i] == y_test[i]
        ax[i].set_title(f"pred␣{final_opt_a[i]}␣/␣true␣{
           y_test[i]}", fontsize=8, color="green" if ok else
           "red")
        ax[i].axis("off")
    plt.tight_layout()
    plt.show()

res_visual(20)
```

```
Prediction results of the first 20 images:
```



Chapter 2

ModelArts Transfer Learning (MobileNetV2)

2.1 Huawei AI Certification: Lab 9.5

ModelArts ExeML Equivalent (Local Image Classification)

Objective: Construct an Image Classification pipeline replicating the ModelArts capability locally. By leveraging Transfer Learning (MobileNetV2), we systematically ingest massive real-world Image Datasets (flowers) and abstract advanced feature arrays accurately and highly computationally efficient!

```
import pathlib
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np

%matplotlib inline
```

```
WARNING: All log messages before absl::InitializeLog() is
   called are written to STDERR
I0000 00:00:1777684688.089842 2694181 cudart_stub.cc:31]
   Could not find cuda drivers on your machine, GPU will not
   be used.
I0000 00:00:1777684688.919096 2694181 cpu_feature_guard.cc
   :227] This TensorFlow binary is optimized to use available
    CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other
   operations, rebuild TensorFlow with the appropriate
   compiler flags.
WARNING: All log messages before absl::InitializeLog() is
   called are written to STDERR
I0000 00:00:1777684690.385648 2694181 cudart_stub.cc:31]
   Could not find cuda drivers on your machine, GPU will not
   be used.
```

1. Cloud Dataset Acquisition & Extraction

Programmatically initiating internal retrieval blocks across HTTPS to safely extract raw Image folders into classification directories.

```
PROJECT_DIR = pathlib.Path.cwd()
DATASET_DIR = PROJECT_DIR / 'data'
DATASET_DIR.mkdir(exist_ok=True)

dataset_url = "https://storage.googleapis.com/download.
   tensorflow.org/example_images/flower_photos.tgz"
archive = tf.keras.utils.get_file(
    'flower_photos.tgz',
    origin=dataset_url,
    extract=True,
    cache_dir=str(DATASET_DIR),
    cache_subdir='',
)
archive_path = pathlib.Path(archive)
candidates = [
    archive_path.parent / 'flower_photos_extracted' / '
       flower_photos',
    archive_path.parent / 'flower_photos',
]
data_dir = next((p for p in candidates if p.exists()), None)
if data_dir is None:
    raise FileNotFoundError(f"Missing flower_photos beside {
       archive_path}")
print("Dataset securely mounted at:", data_dir)
```

```
Downloading data from https://storage.googleapis.com/download
   .tensorflow.org/example_images/flower_photos.tgz
228813984/228813984 -------------------- 7s 0us/step
Dataset securely mounted at: /home/ansinitro/AITU/ai-nn/
   midterm/Labs_notebooks/9.5 ModelArts Lab Guide/data/
   flower_photos_extracted/flower_photos
```

2. Generative Image Loaders & Validations

Using native `image_dataset_from_directory` to mathematically divide training partitions correctly and batching the vectors.

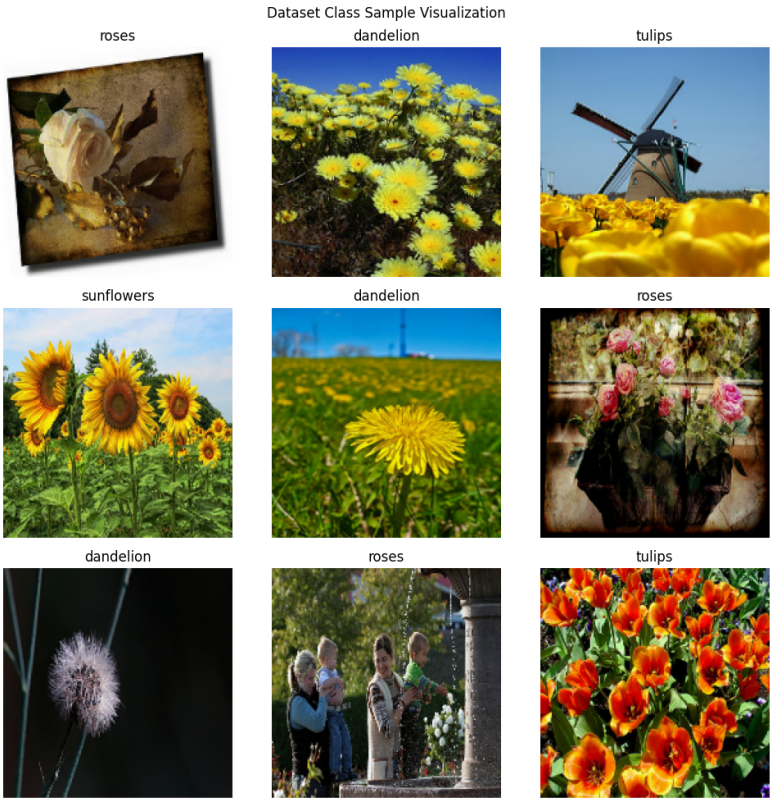
```
IMG_SIZE = 180
BATCH_SIZE = 32
EPOCHS = 10

train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training",
```

```
    seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE
)

class_names = train_ds.class_names
print("Target␣Classification␣Classes:", class_names)
```

```
Found 3670 files belonging to 5 classes.
Using 2936 files for training.
Found 3670 files belonging to 5 classes.
Using 734 files for validation.
Target Classification Classes: ['daisy', 'dandelion', 'roses
   ', 'sunflowers', 'tulips']


E0000 00:00:1777684701.989102 2694181 cuda_executor.cc:1737]
   INTERNAL: CUDA Runtime error: Failed call to
   cudaGetRuntimeVersion: Error loading CUDA libraries. GPU
   will not be used.: Error loading CUDA libraries. GPU will
   not be used.
W0000 00:00:1777684701.989408 2694836 cuda_executor.cc:1755]
   Failed to determine cuDNN version (Note that this is
   expected if the application doesn't link the cuDNN plugin)
   : INTERNAL: cuDNN error: CUDNN_STATUS_INTERNAL_ERROR
W0000 00:00:1777684702.016392 2694181 gpu_device.cc:2365]
   Cannot dlopen some GPU libraries. Please make sure the
   missing libraries mentioned above are installed properly
   if you would like to use GPU. Follow the guide at https://
   www.tensorflow.org/install/gpu for how to download and
   setup the required libraries for your platform.
Skipping registering GPU devices...
```

```
plt.figure(figsize=(10, 10))
for images, labels in train_ds.take(1):
    for i in range(9):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
```

```
plt.suptitle('Dataset Class Sample Visualization')
plt.tight_layout()
plt.show()
```



3. Transfer Learning (MobileNetV2) Architecture Pipeline

Freezing the computationally heavy `imagenet` parameters, mapping our arbitrary classes via pooling structures seamlessly.

```
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().shuffle(1000).prefetch(
   buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights='imagenet'
)
base_model.trainable = False

model = models.Sequential([
    layers.Rescaling(1./255, input_shape=(IMG_SIZE, IMG_SIZE,
        3)),
    base_model,
    layers.GlobalAveragePooling2D(),
```

```
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(len(class_names), activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("Architecture␣successfully␣locked!␣Proceeding␣to␣
   execution␣pipeline...")
```

```
/tmp/ipykernel_2694181/1128898239.py:5: UserWarning: `
   input_shape` is undefined or non-square, or `rows` is not
   in [96, 128, 160, 192, 224]. Weights for input shape (224,
    224) will be loaded as the default.
  base_model = tf.keras.applications.MobileNetV2(


Downloading data from https://storage.googleapis.com/
   tensorflow/keras-applications/mobilenet_v2/
   mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0
   _224_no_top.h5
9406464/9406464 -------------------- 1s 0us/step
Architecture successfully locked! Proceeding to execution
   pipeline...


/home/ansinitro/AITU/ai-nn/midterm/.venv/lib/python3.12/site-
   packages/keras/src/layers/preprocessing/data_layer.py:95:
   UserWarning: Do not pass an `input_shape`/`input_dim`
   argument to a layer. When using Sequential models, prefer
   using an `Input(shape)` object as the first layer in the
   model instead.
  super().__init__(**kwargs)
```

4. Evaluation Loop & Graphical Validation

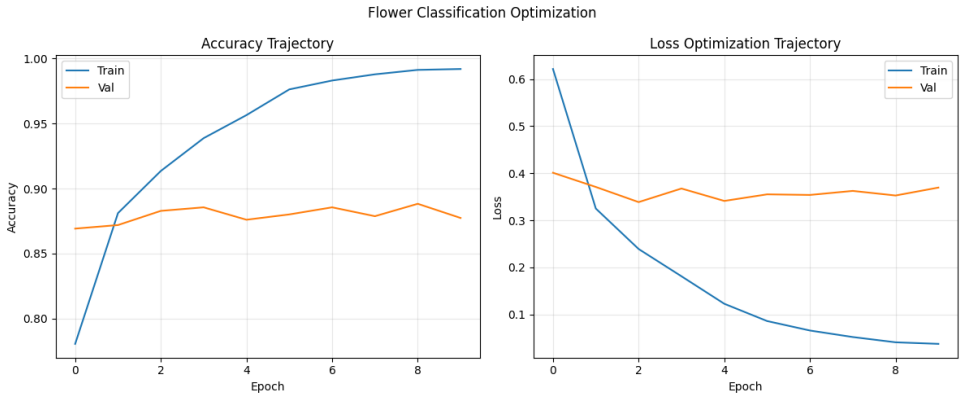
Deploy the execution across configured Epochs and visualize structural accuracy improvements natively.

```
history = model.fit(train_ds, validation_data=val_ds, epochs=
   EPOCHS)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].plot(history.history['accuracy'], label='Train')
axes[0].plot(history.history['val_accuracy'], label='Val')
```

```
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].set_title('Accuracy␣Trajectory')
axes[0].legend()
axes[0].grid(alpha=0.3)

axes[1].plot(history.history['loss'], label='Train')
axes[1].plot(history.history['val_loss'], label='Val')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].set_title('Loss␣Optimization␣Trajectory')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.suptitle('Flower␣Classification␣Optimization')
plt.tight_layout()
plt.show()
```

```
Epoch 1/10


W0000 00:00:1777684711.455531 2694844 cpu_allocator_impl.cc
   :82] Allocation of 99532800 exceeds 10% of free system
   memory.
W0000 00:00:1777684711.610671 2694844 cpu_allocator_impl.cc
   :82] Allocation of 101756928 exceeds 10% of free system
   memory.
W0000 00:00:1777684711.865218 2694844 cpu_allocator_impl.cc
   :82] Allocation of 37324800 exceeds 10% of free system
   memory.
W0000 00:00:1777684711.888296 2694844 cpu_allocator_impl.cc
   :82] Allocation of 37324800 exceeds 10% of free system
   memory.
W0000 00:00:1777684711.925538 2694844 cpu_allocator_impl.cc
   :82] Allocation of 37324800 exceeds 10% of free system
   memory.


92/92 -------------------- 56s 542ms/step - accuracy: 0.7807
   - loss: 0.6213 - val_accuracy: 0.8692 - val_loss: 0.4010
Epoch 2/10
92/92 -------------------- 46s 502ms/step - accuracy: 0.8811
   - loss: 0.3250 - val_accuracy: 0.8719 - val_loss: 0.3708
Epoch 3/10
92/92 -------------------- 81s 497ms/step - accuracy: 0.9135
   - loss: 0.2391 - val_accuracy: 0.8828 - val_loss: 0.3387
Epoch 4/10
92/92 -------------------- 45s 490ms/step - accuracy: 0.9387
   - loss: 0.1812 - val_accuracy: 0.8856 - val_loss: 0.3675
```

```
Epoch 5/10
92/92 -------------------- 47s 511ms/step - accuracy: 0.9564
   - loss: 0.1228 - val_accuracy: 0.8760 - val_loss: 0.3412
Epoch 6/10
92/92 -------------------- 52s 570ms/step - accuracy: 0.9762
   - loss: 0.0861 - val_accuracy: 0.8801 - val_loss: 0.3552
Epoch 7/10
92/92 -------------------- 76s 509ms/step - accuracy: 0.9830
   - loss: 0.0660 - val_accuracy: 0.8856 - val_loss: 0.3539
Epoch 8/10
92/92 -------------------- 45s 494ms/step - accuracy: 0.9877
   - loss: 0.0521 - val_accuracy: 0.8787 - val_loss: 0.3625
Epoch 9/10
92/92 -------------------- 45s 495ms/step - accuracy: 0.9911
   - loss: 0.0410 - val_accuracy: 0.8883 - val_loss: 0.3527
Epoch 10/10
92/92 -------------------- 46s 497ms/step - accuracy: 0.9918
   - loss: 0.0376 - val_accuracy: 0.8774 - val_loss: 0.3695
```

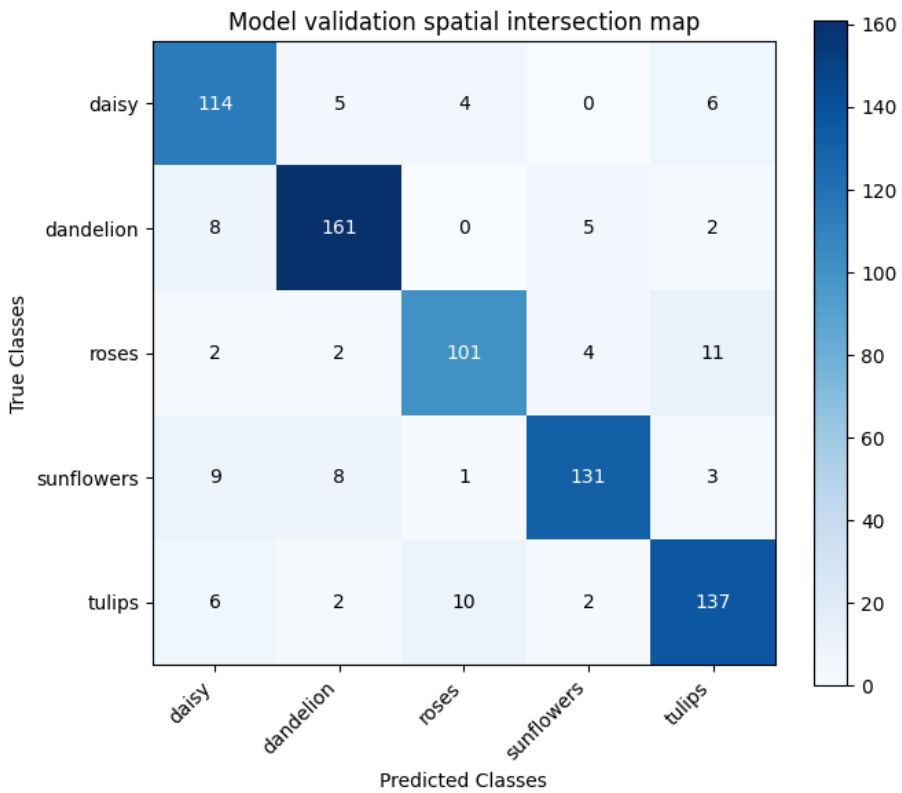


5. Prediction Confusion Matrix

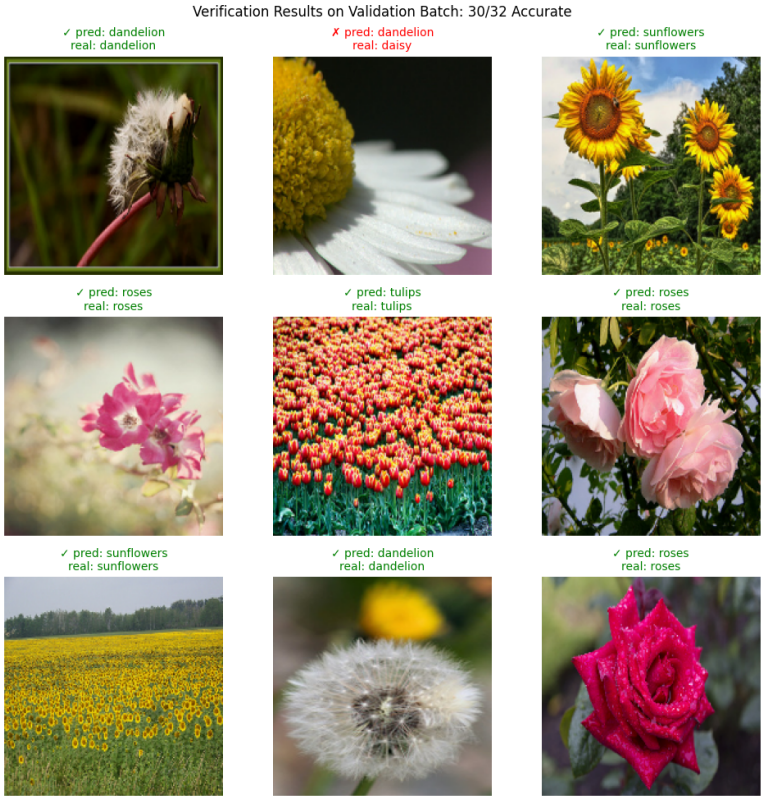
Isolating classification false-positive intersections structurally.

```
all_true = []
all_pred = []
for imgs, lbls in val_ds:
    p = model.predict(imgs, verbose=0)
    all_pred.append(np.argmax(p, axis=1))
    all_true.append(lbls.numpy())
all_true = np.concatenate(all_true)
all_pred = np.concatenate(all_pred)

cm = tf.math.confusion_matrix(all_true, all_pred, num_classes
   =len(class_names)).numpy()

fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(cm, cmap='Blues')
```

```
ax.set_xticks(range(len(class_names)))
ax.set_yticks(range(len(class_names)))
ax.set_xticklabels(class_names, rotation=45, ha='right')
ax.set_yticklabels(class_names)
ax.set_xlabel('Predicted Classes')
ax.set_ylabel('True Classes')
ax.set_title('Model validation spatial intersection map')
threshold = cm.max() / 2
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        ax.text(j, i, str(cm[i, j]), ha='center', va='center'
            , color='white' if cm[i, j] > threshold else '
            black')
plt.colorbar(im, ax=ax)
plt.tight_layout()
plt.show()
```



6. Endpoint Prediction Execution

```
images_batch, labels_batch = next(iter(val_ds))
preds = model.predict(images_batch, verbose=0)
pred_labels = np.argmax(preds, axis=1)
true_labels = labels_batch.numpy()

n_show = min(9, len(images_batch))
```

```
correct_total = int((pred_labels == true_labels).sum())

plt.figure(figsize=(10, 10))
for i in range(n_show):
    plt.subplot(3, 3, i + 1)
    plt.imshow(images_batch[i].numpy().astype("uint8"))
    is_correct = pred_labels[i] == true_labels[i]
    color = 'green' if is_correct else 'red'
    mark = 'v' if is_correct else 'x'
    plt.title(f"{mark}␣pred:␣{class_names[pred_labels[i]]}\
       nreal:␣{class_names[true_labels[i]]}", color=color,
       fontsize=10)
    plt.axis("off")
plt.suptitle(f'Verification␣Results␣on␣Validation␣Batch:␣{
   correct_total}/{len(images_batch)}␣Accurate')
plt.tight_layout()
plt.show()
```